

Каскадни обновявания и изтривания, трансакции, заклучване на данните при трансакция

КАСКАДНИ ОБНОВЯВАНИЯ И ИЗТРИВАНИЯ

Основно понятие

В релационните бази данни таблиците са свързани чрез външни ключове (FOREIGN KEY).

Външният ключ осигурява референтна цялост — гарантира, че:

- не може да има „висящи“ (несъществуващи) връзки
- всяка стойност сочи към реален запис

Нека имаме две таблици:

- coaches (треньори)
- sportgroups (групи)

coaches	sportgroups
CREATE TABLE coaches (id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(100));	CREATE TABLE sportgroups (id INT PRIMARY KEY AUTO_INCREMENT, group_name VARCHAR(100), coach_id INT, FOREIGN KEY (coach_id) REFERENCES coaches(id));
INSERT INTO coaches (name) VALUES ('Иван'), ('Мария');	INSERT INTO sportgroups (group_name, coach_id) VALUES ('Футбол', 1), ('Тенис', 1), ('Баскетбол', 2);

Какво става при изтриване?

```
DELETE FROM coaches WHERE id = 1;
```

Грешка: „Cannot delete or update a parent row“

Има записи в sportgroups, които сочат към този треньор.

Каскадни правила (CASCADE RULES) – тези правила се задават при дефиниране на външния ключ.

ON DELETE RESTRICT (по подразбиране) – забранява изтриването, ако има зависими записи

Използва се когато:

- данните са критични
- не трябва да се губи информация

ON DELETE CASCADE – при изтриване на родител всички свързани записи също се изтриват

FOREIGN KEY (coach_id) REFERENCES coaches(id)

ON DELETE CASCADE

DELETE FROM coaches WHERE id = 1;

Резултат:

sportgroups
Баскетбол (2)

„Футбол“ и „Тенис“ изчезват автоматично

Това може да изтрие много данни наведнъж!

ON DELETE SET NULL – при изтриване връзката се премахва, но записите остават

FOREIGN KEY (coach_id) REFERENCES coaches(id)

ON DELETE SET NULL

Резултат:

group_name	coach_id
Футбол	NULL
Тенис	NULL

Колоната трябва да допуска NULL:

coach_id INT NULL

ON UPDATE CASCADE – при промяна на ключ всички свързани записи се обновяват

FOREIGN KEY (coach_id) REFERENCES coaches(id)

ON UPDATE CASCADE

UPDATE coaches SET id = 10 WHERE id = 2;

Автоматично coach_id = 10

Правило	Какво прави
RESTRICT	Забранява
CASCADE	Изтрива/обновява
SET NULL	Премахва връзката

Проверка за каскадни операции

В MySQL можем да проверим дали има зададени каскадни правила чрез системната база `information_schema`.

ON DELETE CASCADE

```
SELECT table_name
FROM information_schema.referential_constraints
WHERE constraint_schema = 'database_name'
  AND referenced_table_name = 'coaches'
  AND delete_rule = 'CASCADE';
```

Показва таблиците, в които при изтриване се прилага CASCADE.

ON UPDATE CASCADE

```
SELECT table_name
FROM information_schema.referential_constraints
WHERE constraint_schema = 'database_name'
  AND referenced_table_name = 'coaches'
  AND update_rule = 'CASCADE';
```

Показва таблиците, в които при обновяване се прилага CASCADE.

Изтриване при RESTRICT

При RESTRICT (или NO ACTION) НЕ можем да изтрием родителски запис, ако има свързани редове.

Решение: първо премахваме зависимостите

Вариант 1: зануляване на връзката

```
UPDATE sportgroups
SET coach_id = NULL
WHERE coach_id = 1;
```

```
DELETE FROM coaches
WHERE id = 1;
```

Работи само ако колоната позволява NULL.

Вариант 2: изтриване на зависимите редове

```
DELETE FROM sportgroups
WHERE coach_id = 1;
```

```
DELETE FROM coaches
WHERE id = 1;
```

Вариант 3: пренасочване към друг запис

```
UPDATE sportgroups  
SET coach_id = 2  
WHERE coach_id = 1;
```

```
DELETE FROM coaches  
WHERE id = 1;
```

RESTRICT = първо махни зависимостите, после трий

Как ще ги махнеш зависи от логиката:

- триене
- пренасочване
- NULL

ТРАНСАКЦИИ

Трансакция е група от SQL заявки, които се изпълняват като едно цяло.

или всички се изпълняват, или нито една

Кога използваме трансакции – когато имаме повече от една свързана операция

- прехвърляне на пари
- създаване на поръчка
- изтриване на свързани данни

Пример 1: Банков превод

1. намаляваме сумата от сметка А
2. увеличаваме сумата в сметка Б

това е една логическа операция

Пример 2: Онлайн магазин

1. създаваме поръчка
2. добавяме продукти
3. намаляваме наличност

Пример 3: Изтриване на свързани данни

1. изтриваме детайлни записи
2. изтриваме основния запис

Основен синтаксис:

START TRANSACTION;/ BEGIN; -- започва трансакцията

-- заявка 1

-- заявка 2

-- заявка 3

COMMIT; -- записва всички промени

Отмяна на трансакция:

START TRANSACTION;

-- заявки

ROLLBACK; -- отменя всички направени промени

Как работи трансакцията:

Начално състояние:

id	amount
1	1000
2	500

Без трансакция

UPDATE accounts SET amount = amount - 200 WHERE id = 1;

-- грешка

UPDATE accounts SET amount = amount + 200 WHERE id = 2;

резултат:

id	amount
1	800
2	500

грешно състояние

С трансакция:

START TRANSACTION;

UPDATE accounts SET amount = amount - 200 WHERE id = 1;

UPDATE accounts SET amount = amount + 200 WHERE id = 2;

COMMIT;

резултат:

id	amount
1	800
2	700

правилно

При грешка

```
START TRANSACTION;
```

```
UPDATE accounts SET amount = amount - 200 WHERE id = 1;
```

```
-- грешка
```

```
ROLLBACK;
```

резултат:

id	amount
1	1000
2	500

всичко се връща

MySQL използва специален InnoDB log файл, в който записва предишните стойности на данните преди изпълнението на заявките.

Ако ROLLBACK бъде извикан, всички промени се отменят и таблиците остават в предишното си състояние.

COMMIT = запази промените

ROLLBACK = върни промените назад

SAVEPOINT – разделя транзакцията на части, „точка за връщане“ вътре в транзакцията

```
START TRANSACTION;
```

```
UPDATE accounts SET amount = amount - 100 WHERE id = 1;
```

```
SAVEPOINT sp1;
```

```
UPDATE accounts SET amount = amount + 100 WHERE id = 2;
```

```
ROLLBACK TO sp1;
```

```
COMMIT;
```

AUTOCOMMIT – по подразбиране MySQL записва веднага всяка заявка. Когато започнем транзакция, спираме това поведение и решаваме накрая дали да запишем или да върнем промените.

Заклучване на данни при трансакции (LOCKING)

Когато работим с трансакции, базата данни трябва да гарантира, че данните остават консистентни, дори когато има няколко потребители едновременно.

Когато в рамките на трансакция се изпълни UPDATE, INSERT или DELETE, засегнатите редове се заключват. Заклучването остава активно, докато не изпълним:

- COMMIT
- или ROLLBACK

Докато редът е заключен:

- други потребители не могат да го променят
- могат да чакат, докато заключването бъде освободено

START TRANSACTION;

UPDATE customer_accounts

SET amount = amount - 50

WHERE id = 1;

-- няма COMMIT или ROLLBACK

В този момент:

- редът с id = 1 е заключен
- промяната не е окончателна

Друг потребител изпълнява:

SELECT *

FROM customer_accounts

WHERE id = 1;

Той ще види старите стойности, защото промяната още не е потвърдена с COMMIT.

Ако изпълним COMMIT;

- заключването се премахва
- промените стават видими за всички

Поведение при изчакване (timeout):

Ако една заявка чака твърде дълго заключен ред:

- тя може да бъде прекратена
- MySQL автоматично изпълнява ROLLBACK

Това предпазва системата от „забиване“

Принудително заключване с SELECT FOR UPDATE

По подразбиране SELECT не заключва редове, но можем да го направим ръчно:

```
START TRANSACTION;  
SELECT *  
FROM customer_accounts  
WHERE id = 1  
FOR UPDATE;
```

Заклучва реда предварително, така че:

- никой друг не може да го промени
- докато не изпълним COMMIT или ROLLBACK

Видове заключвания

Автоматични заключвания

Създават се автоматично при UPDATE, INSERT и DELETE

- заключват само засегнатите редове
- премахват се при COMMIT или ROLLBACK

Ръчни заключвания

Заклучване на редове

```
SELECT * FROM customer_accounts WHERE id = 1 FOR UPDATE;
```

Заклучване на цяла таблица

```
LOCK TABLE customer_accounts WRITE;
```

```
UPDATE customer_accounts  
SET amount = amount - 50  
WHERE id = 1;
```

```
UNLOCK TABLES;
```

Докато таблицата е заключена:

- никой друг не може да я променя
- достъпът е ограничен